Aprendizagem 2022/23

Homework I – Group XXX

(istxxxxxx, istxxxxxx)

I. Pen-and-paper

2. Pen-and-Paper

1

a)

```
from scipy.stats import multivariate_normal

y1 = [0.24, 0.16, 0.32]
y2 = [0.36, 0.48, 0.72]

mean = [np.mean(y1), np.mean(y2)]
covariance = np.cov(np.vstack((y1, y2)))

mv_normal = multivariate_normal(mean=mean, cov=covariance)

print(f"Multivariable normal for y1 and y2, knowing label is A : mean :{mean}\ncovariance :\n{covariance}")
```

```
Multivariable normal for y1 and y2, knowing label is A : mean :[0.24, 0.52]
covariance :
[[0.0064 0.0096]
 [0.0096 0.0336]]
```

```
from scipy.stats import multivariate_normal

y1 = [0.54, 0.66, 0.76, 0.41]
y2 = [0.11, 0.39, 0.28, 0.53]

mean = [np.mean(y1), np.mean(y2)]
covariance = np.cov(np.vstack((y1, y2)))

mv_normal = multivariate_normal(mean=mean, cov=covariance)

print(f"Multivariable normal for y1 and y2, knowing label is B : mean :{mean}\ncovariance :\n{covariance}")
```

```
Multivariable normal for y1 and y2, knowing label is B : mean :[0.5925, 0.3275]
covariance :
[[ 0.02289167 -0.00975833]
 [-0.00975833  0.03149167]]
```

```
[28] from scipy.stats import multivariate_normal

     y1 = [0.24, 0.16, 0.32, 0.54, 0.66, 0.76, 0.41]
     y2 = [0.36, 0.48, 0.72, 0.11, 0.39, 0.28, 0.53]

     mean = [np.mean(y1), np.mean(y2)]
     covariance = np.cov(np.vstack((y1, y2)))

     mv_normal = multivariate_normal(mean=mean, cov=covariance)

     print(f"Multivariable normal for y1 and y2 : mean :{mean}\ncovariance :\n{covariance}")
```

```
Multivariable normal for y1 and y2 : mean :[0.4414285714285714, 0.41000000000000003]
covariance :
[[ 0.04908095 -0.02106667]
 [-0.02106667  0.03753333]]
```

Aprendizagem 2022/23

Homework I – Group XXX

(istxxxxxx, istxxxxxx)

$P(y_3=0 \wedge y_4=0 \mid y_6=A) = 0$

$P(y_3=0 \wedge y_4=1 \mid y_6=A) = \frac{1}{3}$

$P(y_3=1 \wedge y_4=0 \mid y_6=A) = \frac{1}{3}$

$P(y_3=1 \wedge y_4=1 \mid y_6=A) = \frac{1}{3}$

$P(y_3=0 \wedge y_4=0 \mid y_6=B) = \frac{2}{4}$

$P(y_3=0 \wedge y_4=1 \mid y_6=B) = \frac{1}{4}$

$P(y_3=1 \wedge y_4=0 \mid y_6=B) = \frac{1}{4}$

$P(y_3=1 \wedge y_4=1 \mid y_6=B) = 0$

$P(y_5=0 \mid y_6=A) = \frac{1}{3}$

$P(y_5=1 \mid y_6=A) = \frac{1}{3}$

$P(y_5=2 \mid y_6=A) = \frac{1}{3}$

$P(y_5=0 \mid y_6=B) = \frac{1}{4}$

$P(y_5=1 \mid y_6=B) = \frac{2}{4}$

$P(y_5=2 \mid y_6=B) = \frac{1}{4}$

$P(y_3=0 \wedge y_4=0) = \frac{2}{7}$

$P(y_3=0 \wedge y_4=1) = \frac{2}{7}$

$P(y_5=0) = \frac{2}{7}$

$P(y_5=1) = \frac{3}{7}$

$P(y_3 = 0 \wedge y_4 = 0) = \frac{2}{7}$

$P(y_3 = 0 \wedge y_4 = 1) = \frac{2}{7}$

$P(y_3 = 1 \wedge y_4 = 0) = \frac{2}{7}$

$P(y_3 = 1 \wedge y_4 = 1) = \frac{1}{7}$

$P(y_5 = 0) = \frac{2}{7}$

$P(y_5 = 1) = \frac{3}{7}$

$P(y_5 = 2) = \frac{2}{7}$

$P(y_6 = A) = \frac{3}{7}$

$P(y_6 = B) = \frac{4}{7}$

```
[54] print(mv_normal_A.pdf((0.38,0.52)),
     mv_normal_B.pdf((0.38,0.52)),
     mv_normal_AB.pdf((0.38,0.52)))

     0.9847047567047768 1.9623691378628592 3.6224715484985666
```

```
print(mv_normal_A.pdf((0.42,0.59)),
mv_normal_B.pdf((0.42,0.59)),
mv_normal_AB.pdf((0.42,0.59)))

0.4030709859326153 1.7285705872234585 2.5388068005923157
```

$$P(y_6 | \underline{x}) = \frac{P(\underline{x} | y_6) P(y_6)}{P(\underline{x})} = \frac{P(y_1, y_2 | y_6) P(y_3, y_4 | y_6) P(y_5 | y_6) P(y_6)}{P(y_1, y_2) P(y_3, y_4) P(y_5)}$$

b)

$$P(y_6 = A | \underline{x}_8) = \frac{0,985 \times \frac{1}{3} \times \frac{1}{3} \times \frac{3}{7}}{3,622 \times \frac{2}{7} \times \frac{2}{7}} = 0,159$$

$$P(y_6 = B | \underline{x}_8) = \frac{1,962 \times \frac{1}{4} \times \frac{1}{4} \times \frac{4}{7}}{3,622 \times \frac{2}{7} \times \frac{2}{7}} = 0,237$$

$$P(y_6 = A | \underline{x}_9) = \frac{0,403 \times \frac{1}{3} \times \frac{1}{3} \times \frac{3}{7}}{2,539 \times \frac{2}{7} \times \frac{3}{7}} = 0,123$$

$$P(y_6 = B | \underline{x}_9) = \frac{1,729 \times \frac{1}{4} \times \frac{2}{4} \times \frac{4}{7}}{2,539 \times \frac{2}{7} \times \frac{3}{7}} = 0,596$$

The Bayesian classifier for $\underline{x}_8$ is B and for $\underline{x}_9$ is B.

c) $h_{ML} = \underset{h}{\arg\max}\ P(D|h)$

$P(\underline{x}_8 | y_6 = A) = 0{,}985 \times \frac{1}{3} \times \frac{1}{3} = 0{,}109$

$P(\underline{x}_9 | y_6 = A) = 0{,}403 \times \frac{1}{3} \times \frac{1}{3} = 0{,}045$

Normalization:

$$P(\underline{x}_8 | y_6 = A) = \frac{P(\underline{x}_8 | y_6 = A)}{P(\underline{x}_8 | A) + P(\underline{x}_8 | B)} = 0{,}4716$$

$P(\underline{x}_9 | y_6 = A) = 0{,}1717$

For $\theta < 0{,}1717$ our prediction will be $y_6 = B$ for $\underline{x}_8$ and $\underline{x}_9$ so our accuracy will be 0,5

For $0{,}1717 < \theta < 0{,}4716$ our prediction will be $y_6 = A$ for $\underline{x}_8$ and $y_6 = B$ for $\underline{x}_9$ so our accuracy will be 1

For $\theta > 0{,}4716$ our prediction will be $y_6 = A$ for $\underline{x}_8$ and $y_6 = A$ for $\underline{x}_9$ so our accuracy will be 0,5

R: The thresholds that optimize testing accuracy are between 0,1717 and 0,4716

2

a) First iteration:

Training sets: $\{x_4, x_5, x_6, x_7, x_8, x_9\}$
Validation sets: $\{x_1, x_2, x_3\}$

Second iteration:

Training set: $\{x_1, x_2, x_3, x_7, x_8, x_9\}$
Validation set: $\{x_4, x_5, x_6\}$

Third iteration

Training set: $\{x_1, x_2, x_3, x_4, x_5, x_6\}$
Validation set: $\{x_7, x_8, x_9\}$

Features (in all iterations):

y_2 binarized

y_3, y_4, y_5, y_6

y_1 target variable

b)

D	y_1	y_2	y_3	y_4	y_5	y_6
$\mathbf{x}_1$	0.24	0	1	1	0	A
$\mathbf{x}_2$	0.16	0	1	0	1	A
$\mathbf{x}_3$	0.32	1	0	1	2	A
$\mathbf{x}_4$	0.54	0	0	0	1	B
$\mathbf{x}_5$	0.66	0	0	0	0	B
$\mathbf{x}_6$	0.76	0	1	0	2	B
$\mathbf{x}_7$	0.41	1	0	1	1	B
$\mathbf{x}_8$	0.38	1	0	1	0	A
$\mathbf{x}_9$	0.42	1	0	1	1	B

Hamming distance : # elements in which inputs differ.

$H(x_1, x_4) = 4$
$H(x_1, x_5) = 3$
$H(x_1, x_6) = 3$
$H(x_1, x_7) = 4$
$H(x_1, x_8) = 3$
$H(x_1, x_9) = 4$

$$\hat{z}_1 = \frac{\frac{1}{3} \times 0{,}66 + \frac{1}{3} \times 0{,}76 + \frac{1}{3} \times 0{,}38}{\frac{1}{3} + \frac{1}{3} + \frac{1}{3}} = 0{,}60$$

$H(x_2, x_4) = 2$
$H(x_2, x_5) = 3$
$H(x_2, x_6) = 2$
$H(x_2, x_7) = 4$
$H(x_2, x_8) = 4$
$H(x_2, x_9) = 4$

$$\hat{z}_2 = \frac{\frac{1}{2} \times 0{,}54 + \frac{1}{3} \times 0{,}66 + \frac{1}{2} \times 0{,}76}{\frac{1}{2} + \frac{1}{3} + \frac{1}{2}} = 0{,}6525$$

$H(x_3, x_4) = 4$
$H(x_3, x_5) = 4$
$H(x_3, x_6) = 4$
$H(x_3, x_7) = 2$
$H(x_3, x_8) = 1$
$H(x_3, x_9) = 2$

$$\hat{z}_3 = \frac{\frac{1}{2} \times 0{,}41 + 1 \times 0{,}38 \times \frac{1}{2} \times 0{,}42}{\frac{1}{2} + 1 + \frac{1}{2}} = 0{,}3975$$

$$MAE = \frac{1}{N} \sum_K |z_K - \hat{z}_K| = \frac{1}{3}\left(|0{,}24 - 0{,}6| + |0{,}16 - 0{,}6525| + |0{,}32 - 0{,}3975|\right) =$$

$$= \frac{1}{3}\left(|-0{,}36| + |-0{,}4925| + |-0{,}0775|\right) = 0{,}31$$

Aprendizagem 2022/23

Homework I – Group XXX

(istxxxxxx, istxxxxxx)

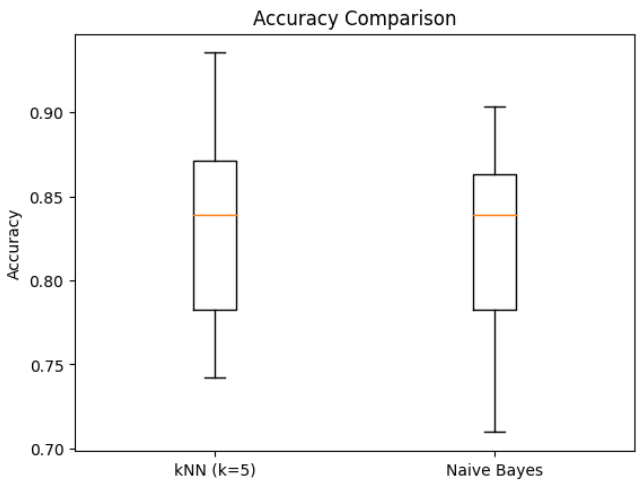
II. Programming and critical analysis

#Loading Dataset

In [ ]:
```
from scipy.io.arff import loadarff
import pandas as pd
import sklearn
import matplotlib.pyplot as plt
import numpy as np



data = loadarff('column_diagnosis.arff')
df = pd.DataFrame(data[0])
df['class'] = df['class'].str.decode('utf-8')

X = df.drop('class', axis=1)
y = df['class']
```

#1.a. Ploting two boxplots with the fold accuracies for each classifier.

In [ ]:
```
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB

knn = KNeighborsClassifier(n_neighbors=5)
nb = GaussianNB()


cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=0)

scores_knn = cross_val_score(knn, X, y, cv=cv)
scores_nb = cross_val_score(nb, X, y, cv=cv)

plt.boxplot([scores_knn, scores_nb], labels=['kNN (k=5)', 'Naive Bayes'])
plt.title('Accuracy Comparison')
plt.ylabel('Accuracy')
plt.show()
```



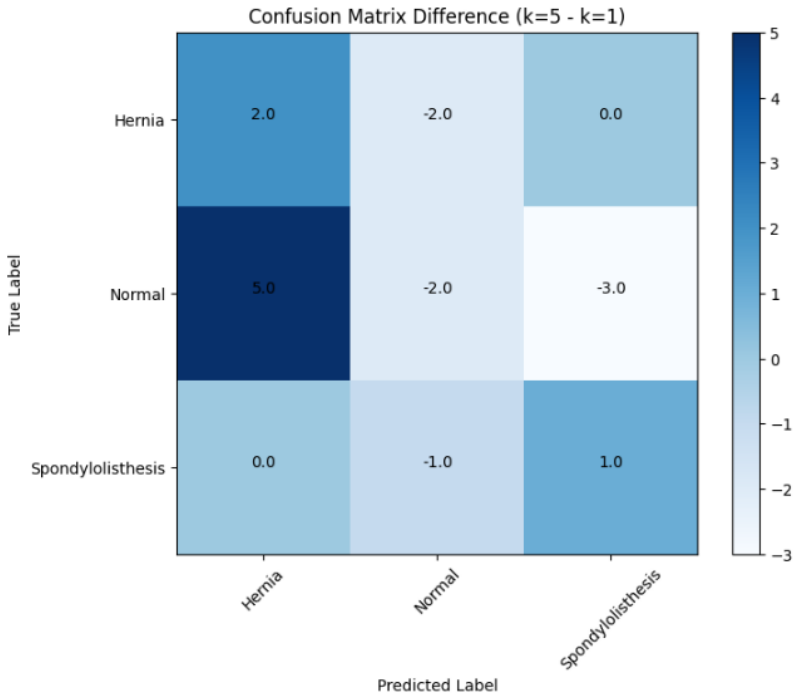
Verifying if kNN is statistically superior to naive Bayes regarding accuracy(using scipy).

In [ ]:
```
from scipy.stats import ttest_rel

t_stat, p_value = ttest_rel(scores_knn, scores_nb)

alpha = 0.05
if p_value < alpha:
    print(f"The difference between kNN and Naive Bayes is statistically significant (p-value = {p_value}).")
else:
    print(f"The difference between kNN and Naive Bayes is not statistically significant (p-value = {p_value}).")
```

```
The difference between kNN and Naive Bayes is not statistically significant (p-value = 0.38085618124128184).
```

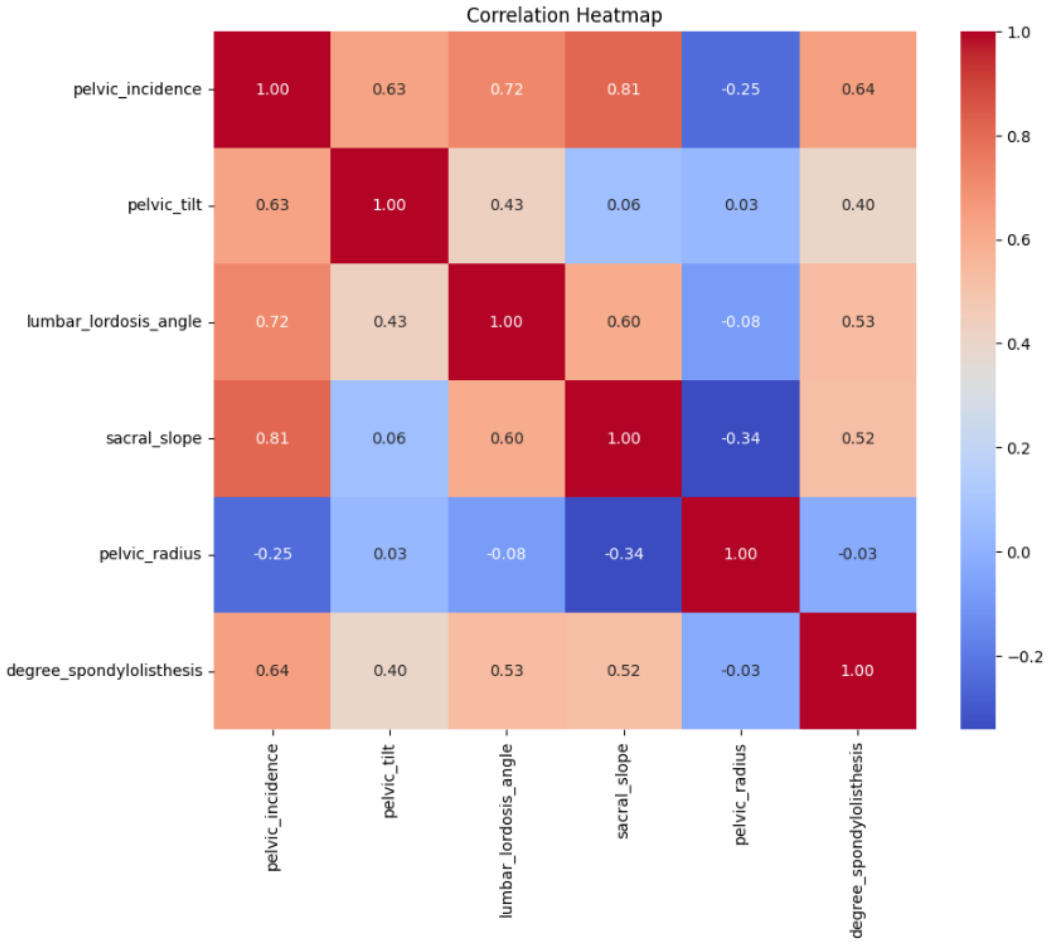
#2. Considering two kNN models, one with k = 1, and another with k = 5. Test them on our cross-validation dataset, and then plotting the difference between the cumulative confusion matrices of the models.

In [ ]:
```
from sklearn.metrics import confusion_matrix

knn_1 = KNeighborsClassifier(n_neighbors=1)
knn_5 = KNeighborsClassifier(n_neighbors=5)

cum_confusion_1 = np.zeros((3, 3))
cum_confusion_5 = np.zeros((3, 3))

for train_index, test_index in cv.split(X, y):
    X_train, X_test = X.iloc[train_index], X.iloc[test_index]
    y_train, y_test = y.iloc[train_index], y.iloc[test_index]

    knn_1.fit(X_train, y_train)
    y_pred_1 = knn_1.predict(X_test)
    cum_confusion_1 += confusion_matrix(y_test, y_pred_1)

    knn_5.fit(X_train, y_train)
    y_pred_5 = knn_5.predict(X_test)
    cum_confusion_5 += confusion_matrix(y_test, y_pred_5)

diff_confusion = cum_confusion_5 - cum_confusion_1

plt.figure(figsize=(8, 6))
plt.imshow(diff_confusion, interpolation='nearest', cmap=plt.cm.Blues)
plt.title('Confusion Matrix Difference (k=5 - k=1)')
plt.colorbar()
tick_marks = np.arange(3)
plt.xticks(tick_marks, knn_1.classes_, rotation=45)
plt.yticks(tick_marks, knn_1.classes_)

for i in range(3):
    for j in range(3):
        plt.text(j, i, str(diff_confusion[i, j]), horizontalalignment="center", color="black")

plt.ylabel('True Label')
plt.xlabel('Predicted Label')

plt.show()
```



#Comment As we can see, the differences between the two models are not very noticeable. However, we can still observe minor discrepancies. Our k=5 model correctly identified two additional individuals with hernias and one additional individual with spondylolisthesis compared to the k=1 model. However, it failed to correctly identify two more normal individuals, than other model did. Additionally, our k=5 model falsely classified five individuals with hernias as normal, which is five more than the k=1 model.

In [ ]:
```
import seaborn

correlation_matrix = df.corr()
plt.figure(figsize=(10, 8))
seaborn.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f")
plt.title("Correlation Heatmap")
plt.show()
```



#3. Identifying three possible difficulties of naive Bayes when learning from the given dataset.

1. As we can see on the heatmap, the non-target features have correlations between them. Some of them are quite strong, for example, the correlation between sacral slope and pelvic incidence (0.81). So the feature independence assumption used by Naive Bayes can affect negatively preformance of our model.
2. Continuous numeric features pose a significant challenge. Many features in our dataset are continuous, and this poses a difficulty for our problem. In order to make predictions, the model will need to calculate the probability of features having specific values. While Naive Bayes works well with classes, for numeric values, it requires determining the probability density function for that feature. This function is usually unknown, and attempting to guess it could result in lower predictive power for our model.
3. Zero frequencies are also a problem we should consider. As we mentioned earlier, our model needs to calculate the probability of features in order to predict a target feature. Therefore, calculating a probability for a value never seen in the training dataset, or for a value that falls outside the observed range, could also result in lower predictive power.

```
In [ ]:
```

END